Camera Effects

Contents

- Setup
- Camera Shake
- Hitstop
- Impact Frames
- The Coordinator
- Gotchas

The hit feedback that sells big moments, mostly the Special Shot:

- **Camera shake** (`CameraEffects`)
- **Hitstop** (`HitstopController`): a short freeze.
- **Impact frames** (`ImpactFrameController`): the anime-style full-screen flash.
- **The coordinator** (`CameraEffectCoordinator`): decides when the three play for the Special Shot.

Each effect only knows how to play itself. The coordinator decides timing. Anything can also call an effect directly.

See also: Special Shot, Shaders & VFX (the Impact Frame shader), Interface & Settings (Camera Shake and Impact Frames settings).

Setup

1. On the player's camera rig, add **CameraEffects** and assign the **Player Camera**.
2. In the **SceneEssentialsBundle**, add **HitstopController**, **ImpactFrameController** and **CameraEffectCoordinator** (one of each).
3. Make sure the Global Volume profile has an **Impact Frame** override, and the Impact Frame renderer feature is on the renderer.

Tune timing on the coordinator, freeze length on the HitstopController, and the look on the Impact Frame Volume Component.

Camera Shake

Wobbles the camera's position and roll using Perlin noise. There are two kinds, which add together:

- **Impact shake**: starts strong and fades out over its duration. A new shake restarts it.
- **Sustained shake**: stays at a set strength until changed or stopped. Can be updated every frame to build smoothly.

Field	What it does
Default Shake Intensity / Duration	Used by `Shake()` with no arguments.
Shake Position Scale	Max position offset.
Shake Roll Scale	Max roll in degrees.
Shake Frequency	Wobble speed.

```
CameraEffects.Instance?.Shake();                // defaults
CameraEffects.Instance?.Shake(0.5f, 0.3f);      // intensity, duration
CameraEffects.Instance?.SetSustainedShake(0.2f); // call each frame to build
CameraEffects.Instance?.StopSustainedShake();
```

Currently used by the coordinator, by `PlayerStatus` when the player takes damage (scaled by damage), and by `PlayerDash` when dashing through an enemy staggers it.

Shake is skipped when the player's **Camera Shake** setting is off.

> **In code:** `CameraEffects` runs late (`[DefaultExecutionOrder(100)]`, in `LateUpdate`) so it applies after other camera scripts. It removes last frame's roll before adding the new one so roll never builds up, and resets the camera's **local position** every frame before adding the offset.

Hitstop

Sets `Time.timeScale` to 0 for a short real-time duration, then puts it back. Everything on scaled time freezes. Anything on unscaled time (like the impact frame) keeps going.

Field	What it does
Duration	Freeze length at full strength. 0 = off.
Min Duration	Shortest a weaker freeze can be.

```
hitstop.Play();        // full strength
hitstop.Play(0.6f);    // 60% of Duration (never below Min Duration)
hitstop.PlayFor(0.2f); // exact seconds
```

Calling it during a freeze restarts the timer. `HitstopController.IsActive` is true while frozen.

Pausing: hitstop doesn't play while paused, and pausing mid-freeze ends it (the pause menu takes over the time scale).

Impact Frames

Plays the Impact Frame post-process as a short sequence of rendered frames, centred on a point on screen:

Phase	Field	Shows
Flash	**Flash Frames** (0 = skip)	Solid colour
Hold	**Hold Frames**	The full effect: two-tone scene, speed lines, jitter
Tail	**Tail Frames** (0 = skip)	Lines and jitter over the normal scene

During the sequence the line pattern re-rolls **Steps Per Second** times a second (7 matches the original Shader Graph).

```
        impactFrameController.Play(worldPosition);
```

The world position is projected onto the screen as the centre (screen centre if it's behind the camera). Calling it again restarts at the new position.

It's skipped when the **Impact Frames** setting is off, and hidden while paused (it carries on after).

> **In code:** the controller finds the Volume whose profile has an `ImpactFrameVolumeComponent` (or use **Impact Frame Volume**) and writes to its **`sharedProfile`**. It only changes the layer toggles, focal point and seed. Frames are counted with `Time.frameCount` and the re-roll uses unscaled time, so it plays through hitstop.

The Coordinator

Turns Special Shot results into feedback.

While charging (both buttons held, charge ready): a sustained shake that builds from **Hold Shake Start Intensity** to **Max Intensity** over **Build Time**. It stops as soon as the player lets go, reloads or pauses.

On hits: with **Per Target** on, this plays once for each target the shot hit, closest first. Each target is a little weaker (**Falloff**):

1. Hitstop.
2. Impact frame at the hit point (with the freeze, or after it if **Impact During Hitstop** is off).

3. Impact shake as the freeze ends.
4. Wait **Target Delay**, then the next target.

The result is a quick chain of punches through each enemy the shot passes.

On a miss: no hitstop. Optional weaker shake (**Shake On Miss**, **Miss Shake Scale**) and optional impact frame (**Impact On Miss**).

A new Special Shot cancels any chain still playing, and so does pausing.

> **In code:** it listens to `WeaponEvents.ShotFired(Special)` to start collecting, stores each `SpecialHit` from `ShotResolved`, then starts the chain in `LateUpdate` once every result from that frame is in. All references are found automatically if left empty.

Adding feedback for another event (e.g. a boss phase change): follow the same pattern in its own script or in the coordinator. Listen for the event, then play the effects in order. Keep one place in charge of each moment rather than several scripts each playing effects.

Gotchas

- **Anything that should keep moving during hitstop must use unscaled time.**
- **Don't move the camera's local position from other scripts.** CameraEffects resets it every frame. Rotation is fine.
- **The impact frame's two-tone uses camera normals**, which sprites don't write.
- **One coordinator, hitstop and impact frame controller per scene.**